

1 3.A - Architecture TCP/IP

1.1 Modèle OSI et Architecture TCP/IP

Qu'est-ce que l'architecture TCP/IP ? C'est l'ensemble des protocoles qui permettent de faire fonctionner le réseau Internet. On va revenir sur ce qu'est un protocole rapidement. Internet est le diminutif de **Inter-Networking** qui signifie Interconnexion de réseaux. C'est donc ce grand réseau mondial qui interconnecte la plupart des réseaux informatiques. Ce réseau permet d'échanger des données, des fichiers et d'utiliser des applications en ligne qui permettent aux utilisateurs d'Internet de communiquer. Dans cette partie, nous allons donc voir rapidement le rôle de chacun de ces protocoles qui font fonctionner Internet. Sachant que dans d'autres parties du cours, nous reviendrons un peu plus en détails sur certains d'entre eux.

D'où vient ce nom d'architecture TCP/IP ? Parce que les deux principaux protocoles d'Internet, c'est-à-dire ceux qui sont un peu au cœur de l'architecture, sont les protocoles IP et TCP. Nous allons voir pourquoi ils jouent un rôle central. Avant de décrire l'architecture, il faut rappeler ce qu'est un protocole. Il y a des protocoles dans le monde des réseaux, mais aussi entre les humains, par exemple. Un **protocole** est ce qui permet à deux entités d'échanger des informations. Quand on veut faire cela, il faut préciser :

- Quelles sont les informations à échanger ?
- Comment les échanger ?
- Avec quelles règles et quels formats de messages ?

Pour mettre en œuvre un protocole, des algorithmes vont s'exécuter soit dans du matériel, soit dans du logiciel, côté émetteur et récepteur.

Une architecture réseau est un ensemble de protocoles qui coopèrent les uns avec les autres pour rendre le service attendu par l'utilisateur du réseau. Chaque protocole rend lui-même un service : il a une fonction bien identifiée, un rôle dans l'architecture. Cette fonction est toujours répartie sur au moins deux entités qui communiquent. Par exemple, l'appelant et l'appelé ou l'émetteur et le récepteur, c'est-à-dire deux téléphones ou un client et un serveur ou deux cartes réseaux ou deux routeurs sur Internet, etc. Pour chaque protocole, il faut donc définir précisément son rôle, sa fonction et donc l'algorithme qu'il va devoir exécuter de chaque côté de la communication, ainsi que le format des messages échangés permettant la réalisation de l'algorithme.

Nous pouvons maintenant décrire l'architecture TCP/IP qui régit le fonctionnement d'Internet.

Sur cette architecture TCP/IP, vous voyez qu'il y a en gros trois blocs. Le **bloc applications** en haut, qui contient tous les protocoles applicatifs d'Internet. Ce sont eux qui permettent aux applications de s'exécuter en échangeant des données sur Internet. Ces protocoles s'exécutent dans des processus utilisateurs appelés **Clients ou Serveurs**, c'est-à-dire des programmes, comme par exemple un navigateur Web ou un terminal ou un logiciel pour faire du courrier électronique ou un outil pour transférer des fichiers. Donc c'est bien un programme que lance l'utilisateur d'Internet pour pouvoir accéder au service qu'il souhaite. Ce programme va s'exécuter côté client et côté serveur. Dans la suite, nous décrirons plus en détails le modèle Client/Serveur qui régit le fonctionnement d'Internet. Le client est celui qui demande quelque chose à un serveur qui se trouve à distance sur Internet. L'utilisateur se trouve donc sur le client. Le serveur est celui qui va répondre aux demandes de l'utilisateur. Par exemple, un serveur Web va répondre aux requêtes d'un navigateur Web, c'est-à-dire qu'il va renvoyer la page Web et les objets associés correspondant à ce qui est mis dans l'URL du navigateur Web.

Le **bloc logiciel** fait l'intermédiaire entre les applications qui font des demandes sur Internet et le matériel qui va permettre la transmission physique des messages échangés. Ce bloc correspond aux protocoles qui vont s'exécuter dans le système d'exploitation d'un équipement terminal (ordinateur, téléphone, tablette, etc.) ou d'un équipement intermédiaire (commutateur, routeur, borne Wi-Fi, box, etc.). Ce bloc regroupe le cœur d'Internet, c'est-à-dire les principaux protocoles que sont TCP, UDP et IP. Ces **protocoles de transfert** permettent de transférer les données. Dans ce même bloc figurent les **protocoles de contrôle** comme ARP ou ICMP. Ils permettent de faire des tests ou de faire fonctionner Internet. Par exemple, ICMP permet des tests avec l'outil **ping** ou **traceroute** en envoyant une requête à un destinataire afin qu'il réponde. Cela permet de vérifier que les échanges fonctionnent bien, de voir par où le message est passé, de mesurer le temps aller-retour, etc.

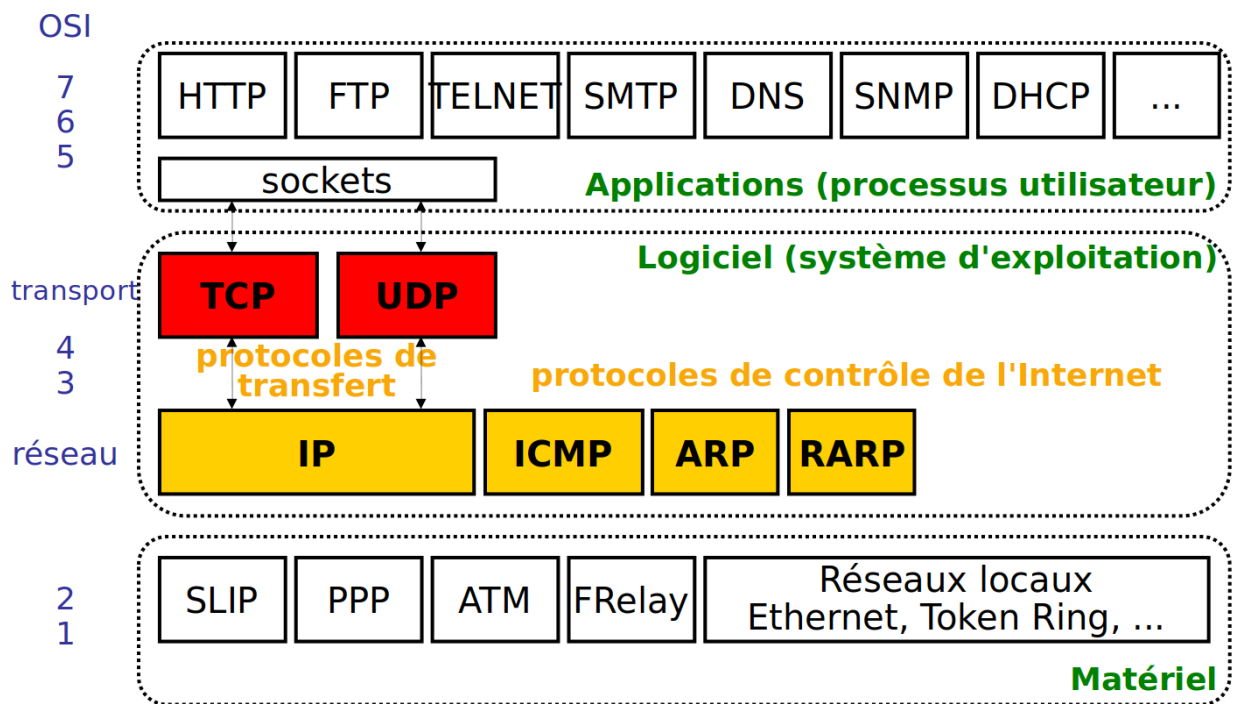


Figure 1: architecture OSI

Le dernier bloc est le **bloc matériel** qui permet d'envoyer les données de manière physique sur un réseau. Quand on parle du matériel, il s'agit généralement d'une carte réseau, souvent de type Ethernet ou Wi-Fi. Les protocoles de ce bloc correspondent aux différents types de liaisons et permettent d'envoyer un signal depuis une carte réseau à une autre carte réseau.

Sur la gauche du schéma figurent les correspondances avec le **modèle OSI**. Ce modèle OSI est un modèle théorique qui décrit le fonctionnement d'une architecture réseau. Il est composé de sept couches. Les couches un et deux correspondent aux couches physique et liaison réalisées par la carte réseau. La couche réseau, qui permet d'acheminer les paquets vers un destinataire et donc de trouver le chemin pour aller jusqu'à la destination, est réalisée sur Internet par le protocole IP. Juste au-dessus, la couche transport (couche 4) d'Internet est réalisée soit par TCP soit par UDP selon le choix et les besoins de l'application.

La particularité de l'architecture TCP/IP est qu'au lieu d'être en 7 couches comme le modèle OSI le préconise, les couches 5, 6 et 7 sont fusionnées en une seule couche appelée la **couche application**. Cependant, certaines applications, telles que **NFS** pour le partage de fichiers, utilisent réellement les couches 5, 6 et 7 du modèle OSI.

Nous allons maintenant décrire les principaux protocoles d'Internet en commençant par les protocoles applicatifs.

1.2 Principaux protocoles applicatifs

Le protocole **HTTP** (HyperText Transfer Protocol) permet de faire dialoguer un navigateur web et un serveur Web. Il permet de transférer des pages Web et les objets qui y sont incorporés, tels que des images, des vidéos, etc.

Le protocole **FTP** (File Transfer Protocol) permet de transférer des fichiers entre un client et un serveur. Le serveur FTP héberge des fichiers et le client peut déposer ou récupérer des fichiers. Ce protocole permet également de créer des dossiers, de supprimer des dossiers, de renommer des fichiers et de gérer le système de gestion de fichiers sur un ordinateur.

Le protocole [telnet](#) permet de faire de la connexion à distance, c'est-à-dire d'exécuter des commandes sur une machine distante. Le client telnet est généralement un terminal qui sert à exécuter des commandes. Le rôle du serveur Telnet est d'exécuter ces commandes à distance.

D'autres protocoles permettent de faire de la connexion à distance, comme [rlogin](#) et, surtout maintenant, le protocole [SSH](#), qui permet de faire la connexion à distance de manière sécurisée, c'est-à-dire avec le chiffrement des communications.

Le protocole [SMTP](#) (Simple Mail Transfer Protocol) permet d'envoyer du courrier électronique, souvent appelé mail ou email. Pour la réception du courrier électronique, il y a d'autres protocoles, tels que [POP](#) et [IMAP](#), sachant que maintenant c'est IMAP qui est le plus largement utilisé. IMAP (comme POP) permet à un client mail (comme par exemple Outlook ou Thunderbird et encore plein d'autres) de récupérer ces courriers électroniques sur un serveur afin de les lire. Le client mail peut aussi envoyer des courriers électroniques grâce au protocole SMTP.

[DNS](#) (Domain Name System) est un protocole un peu particulier. Il s'agit d'un protocole applicatif comme les autres, mais il ne correspond pas à une véritable application en lien avec un utilisateur. En revanche, il est utilisé par toutes les « vraies » applications sans que l'utilisateur en soit averti. Le DNS est vraiment au cœur du fonctionnement d'Internet, car s'il ne fonctionne pas, les autres applications deviennent inaccessibles. Le DNS sert à faire la correspondance entre les noms des machines sur Internet (comme <https://www.education.gouv.fr/>) et leurs adresses Internet ou adresses IP (comme 152.199.21.182). Ainsi, pour n'importe quelle application, le DNS permet à l'application cliente de trouver l'adresse IP du serveur auquel elle veut s'adresser.

Il est donc essentiel car les utilisateurs d'Internet mémorisent le nom des serveurs et non pas leur adresse IP, alors que cette dernière est nécessaire pour trouver le serveur. Le DNS est en quelque sorte le carnet d'adresses ou l'annuaire des noms de serveurs sur Internet, comme vous pouvez avoir un annuaire des numéros de téléphone auxquels vous associez des noms également.

Le protocole [SNMP](#) (Simple Network Management Protocol) n'est pas utilisé directement par les utilisateurs d'Internet. Il est utilisé dans l'administration des réseaux pour échanger des informations entre les équipements d'un réseau. Par exemple, il permet de faire de la surveillance du réseau, de la détection de pannes, du suivi de la charge du réseau et de son fonctionnement. Il permet donc à un administrateur de réseau d'avoir des outils de supervision du réseau.

Pour terminer sur le bloc application, présentons les [sockets](#). Il ne s'agit pas d'un protocole mais d'une bibliothèque, au sens ensemble de fonctions, disponible dans presque tous les langages de programmation *généralistes* (C, Python, Java, ...), qui permet à une application cliente ou serveur de faire ses demandes d'envoi ou de réception de messages en utilisant le protocole TCP ou UDP. Nous présenterons les sockets en détails dans une autre partie du cours.

Passons maintenant à la description des protocoles qui sont au cœur du fonctionnement d'Internet : les protocoles de transport (TCP ou UDP) et le protocole principal d'Internet (IP).

1.3 Protocoles TCP et UDP

Quel est le rôle de **TCP** (***T**ransmission **C**ontrol **P**rotocol*) ? C'est principalement de faire la **fiabilité** comme nous l'avons vu en détail [ici](#). La fiabilité est ce qui permet de s'assurer que toutes les données qui arrivent à destination sont exactement celles qui ont été envoyées au départ, avant de traverser le réseau. Pourquoi peut-il y avoir une différence ? Quand on envoie des messages dans un réseau, ils peuvent être perdus ou subir des erreurs. Une erreur est une donnée qui a changé de valeur pendant sa transmission. Chaque donnée transférée est une séquence binaire constituée de zéros et de uns. Quand on transfère des données sur un réseau, on se ramène toujours à une séquence binaire. Une erreur se produit si, pendant le transfert, un 0 est devenu un 1 ou un 1 est devenu un 0. Quand les données arrivent à destination, selon les besoins de l'application, il peut être nécessaire de vérifier qu'elles ne contiennent pas d'erreur. Il peut également être utile de détecter des pertes éventuelles de données. Ainsi, pour certaines applications, il peut être nécessaire de retransmettre les données erronées ou perdues. Une perte est une donnée qui n'arrive pas à destination. Le rôle principal de TCP est donc de mettre en œuvre la fiabilité, c'est-à-dire de faire

la détection des pertes et des erreurs et de retransmettre tous les messages qui ont subi des erreurs ou des pertes. La plupart des applications d'Internet, à l'exception des applications multimédias, ont besoin de la fiabilité et utilisent donc TCP pour transmettre leurs données. C'est le cas du transfert d'une page Web, d'un courriel ou d'un fichier, mais aussi pour l'exécution d'une commande à distance avec `telnet` ou `ssh`.

Pour réaliser la fiabilité, TCP a besoin du mode connecté. Un client, avant d'échanger des données avec un serveur, doit s'assurer qu'il est disponible et en mesure de dialoguer avec lui. Pour cela, il doit faire une demande d'ouverture de connexion au serveur au début de la communication et la connexion doit être fermée une fois les échanges terminés.

TCP est un protocole complexe qui est décrit dans de nombreuses RFC (par exemple les RFC 793, RFC 1122, RFC 1323, RFC 2018, RFC 2581). Les RFC sont les documents normatifs qui décrivent le fonctionnement des protocoles sur Internet. Et donc il y a plusieurs RFC qui sont définis pour TCP, tellement ce protocole est complexe.

Il transporte un flux d'octets qui provient de l'application. Le principe est que l'application lit ou écrit des octets dans ce qu'on appelle la **socket**, c'est-à-dire des zones mémoire pour envoyer et recevoir des données. Et ensuite il s'occupe de faire que tout arrive dans l'ordre, sans erreur et sans perte, et réalise les retransmissions si nécessaire, au bout d'un certain temps, dans le cas où les accusés de réception n'arrivent pas.

TCP réalise également ce qu'on appelle le **contrôle de flux**. C'est ce qui permet de faire en sorte que les messages ne sont pas envoyés si le récepteur n'est pas prêt à recevoir. Par exemple, si un serveur est bloqué pour une raison ou une autre et ne peut plus recevoir, le contrôle de flux va demander au client d'arrêter d'envoyer des données pour éviter qu'elles ne soient perdues. Le **contrôle de congestion**, c'est ce qui permet à TCP de ne pas envoyer des données dans le réseau s'il y a une congestion, c'est-à-dire à un point d'encombrement dans le réseau. Le principe est que, dès lors que TCP détecte une perte, c'est-à-dire un message qui n'ont pas été acquitté, il considère qu'il y a une congestion dans le réseau et donc il va ralentir ses émissions.

Ainsi, comme nous le verrons [plus loin](#), l'en-tête TCP doit donc contenir de quoi mettre en œuvre la fiabilité (numéro des messages envoyés et reçus, checksum, contrôle de flux) et un moyen permettant de distinguer des messages spécifiques à TCP comme l'ouverture/fermeture de connexion ou l'acquiescement. L'en-tête TCP doit contenir également l'identifiant de l'application cliente et serveur c'est-à-dire les numéros de port source et destination comme expliqué ci-après.

Le protocole **UDP** (*User Datagram Protocol*) est un protocole plus simple qui n'assure pas la fiabilité des transferts. Il n'a donc pas besoin de dérouler tout un algorithme complexe qui est exécuté par TCP pour faire les retransmissions en cas d'erreur ou de perte. UDP est un protocole très simple qui fait le lien entre l'application et le protocole IP. Contrairement à TCP, il permet d'envoyer les données d'une application cliente ou serveur de manière simple et rapide, sans s'assurer de la fiabilité. UDP fait tout de même la détection d'erreur lors de la réception de chaque message en calculant une somme de contrôle. Si une erreur est détectée, le message est détruit mais aucune retransmission n'est mise en œuvre. Contrairement à TCP, UDP n'est pas en mode connecté donc il envoie du client vers le serveur ou inversement sans demander l'accord de l'autre partie et sans se soucier des problèmes liés à des pertes de paquets ou à des erreurs.

Les applications qui utilisent UDP sont celles qui peuvent s'affranchir de la fiabilité, principalement les applications multimédia. Quand on fait de la visioconférence, du streaming, quand on regarde des vidéos, quand on transporte du son ou de la musique, l'oreille et l'œil humain ne sont pas sensibles à quelques erreurs ou pertes. Ces applications sont donc tolérantes aux pertes, mais ont besoin d'un bon débit, ce qui explique pourquoi elles utilisent UDP. Il y a d'autres applications qui ne sont pas multimédia et qui utilisent UDP, le DNS par exemple, car il a besoin de rapidité et de réactivité et qu'il transporte des petits messages. Cela n'empêche pas UDP de faire la détection d'erreurs quand même et donc de supprimer les réponses qui pourraient contenir des erreurs. Dans ce cas, l'application, ici le DNS, ne va pas recevoir de réponse à sa demande et va donc refaire sa requête DNS au serveur. Voilà donc une application qui peut utiliser UDP en faisant elle-même la reprise sur erreur, c'est-à-dire la retransmission en cas d'erreur ou de perte qu'elle aurait détectée elle-même.

Enfin, TCP et UDP ont un rôle commun : ils font le lien entre les applications et le réseau. Ils permettent l'envoi et la réception de messages à travers la librairie *socket* présentée plus haut. Pour faire ce lien avec les applications, ils ont tous les deux un rôle essentiel qui est d'aiguiller les messages qui arrivent ou qui partent vers le bon programme utilisateur, c'est-à-dire le navigateur Web ou l'outil de courrier électronique ou le terminal pour la connexion à distance, etc. Ainsi, TCP et UDP vont gérer l'aiguillage des messages qui sortent ou qui rentrent de la machine pour aller/venir sur Internet. Comment font-ils ça ? En utilisant ce qu'on appelle les **numéros de port** qui permettent de donner un numéro unique à chaque programme client ou serveur sur une machine donnée. Par exemple, par défaut, le numéro de port d'un serveur Web est le port 80.

Nous reviendrons sur les numéros de ports plus loin. Pour l'instant, vous pouvez juste retenir qu'un numéro de port permet d'identifier de manière unique sur une machine donnée un programme client ou serveur qui utilise Internet.

Ainsi, comme nous le verrons [plus loin](#), l'en-tête UDP doit donc contenir uniquement les numéros de port source et destination et un checksum permettant de faire la détection d'erreur. L'entête UDP est petite et simple ce qui permet d'être plus rapide et plus efficace.

1.4 Protocole IP

Le protocole [IP](#) (*Internet Protocol*) est le cœur de l'architecture d'Internet. Son rôle est d'acheminer chaque message depuis un émetteur vers un récepteur, c'est-à-dire de trouver le chemin, par exemple pour aller de Lyon jusqu'à un serveur qui se trouverait à New York. Ce rôle est évidemment essentiel, puisque c'est bien l'essence même d'un réseau que de savoir par où faire circuler les informations qui traversent le réseau.

Pour trouver le chemin, le protocole IP met en œuvre le **routing** des paquets grâce aux routeurs, qui sont les équipements intermédiaires en charge de l'acheminement des paquets. La topologie d'Internet étant très complexe et très dynamique, nous pouvons considérer qu'elle est inconnue des routeurs. Chaque routeur connaît ses voisins mais ne connaît pas la route complète pour aller de n'importe quelle source à n'importe quelle destination sur Internet. Ainsi, le routage se fait **de proche en proche** : pour n'importe quelle destination à atteindre, chaque routeur doit connaître le routeur suivant appelé **prochain saut** ou *next hop*. En revanche, il ne connaît pas le routeur suivant du suivant et encore moins le chemin complet.

Les routeurs se basent sur les adresses IP pour prendre des décisions de routage. Chaque paquet IP est acheminé vers sa destination en fonction de l'adresse IP destination contenue dans l'en-tête du paquet. L'adresse IP permet d'identifier de manière unique chaque machine sur Internet, un peu comme un numéro de téléphone pour les utilisateurs du réseau téléphonique. Il y a plusieurs formats d'adresses IP qui cohabitent sur Internet : les adresses IP version 4 (IPv4) et les adresses IP version 6 (IPv6). L'adresse **IPv4** fait 4 octets et s'écrit sous la forme de 4 nombres décimaux compris entre 0 et 255 séparés par un point (par exemple 132.227.71.30). L'adresse **IPv6** fait 16 octets et s'écrit sous la forme de 8 hextets (1 hextet = 16 bits) en hexadécimal séparés par **:** (par exemple 2001:0db8:3c4d:0015:0000:d234:3eee:0001 qui peut être raccourcie en 2001:db8:3c4d:15::d234:3eee:1). Nous reviendrons sur ces formats [un peu plus loin](#).

Un autre rôle moins connu du protocole IP est la **fragmentation**. Il s'agit d'adapter la taille des paquets IP à la taille maximale des trames sur chaque liaison traversée c'est-à-dire de vérifier que les paquets IP ne sont pas trop gros pour être encapsulés dans une trame et si cela n'est pas le cas de couper (fragmenter) le paquet IP en plusieurs morceaux (fragments) pour que chaque fragment respecte la taille maximale de la trame. Nous verrons en détails ce mécanisme un peu plus loin. On appelle **MTU** (*Maximum Transmission Unit*) d'une liaison, la taille maximale du paquet IP qu'une trame peut contenir. Par exemple, sur une liaison de type Ethernet, la MTU est de 1500 octets car la trame a une taille maximale de 1518 octets avec un en-tête + en-queue Ethernet de 18 octets.

Comme nous le verrons [plus loin](#), l'en-tête IP doit donc contenir a minima la version du protocole IP (4 ou 6), les adresses IP source et destination, les informations nécessaires à la fragmentation et l'identification du contenu du paquet (TCP, UDP ou ICMP).

Après la description des protocoles de transfert de données que sont TCP, UDP et IP, passons aux protocoles de contrôle de l'Internet : ICMP, ARP, RARP et DHCP.

1.5 Protocole ICMP

ICMP (***Internet Control Message Protocol***) permet de faire des tests de connectivité sur le réseau. C'est lui qui permet de faire ce qu'on appelle les **ping**. Il permet également de faire les **traceroute**. La commande **traceroute** cherche tous les routeurs intermédiaires qui permettent d'aller jusqu'à la destination. Elle affiche leurs adresses IP et le temps aller-retour pour les atteindre.

1.6 Protocoles ARP et RARP

Le protocole **ARP** (***Address Resolution Protocol***) est typiquement un protocole essentiel au fonctionnement d'Internet mais qui est invisible pour les utilisateurs, un peu comme le protocole DNS. ARP fait le lien entre les adresses IP utilisées pour trouver une machine sur Internet et les adresses des cartes réseaux utilisées pour déterminer l'émetteur/récepteur d'un signal.

Les adresses des cartes réseaux s'appellent les **adresses MAC** ou les **adresses physiques** et vous sont présentées [ici](#). Ces adresses MAC sont essentielles pour transférer le signal d'une carte réseau à une autre carte réseau. En particulier, lorsqu'une carte réseau reçoit un signal, l'adresse MAC lui permet de savoir si ce signal lui est destiné ou non. Les adresses MAC sont utilisées sur chaque liaison traversée sur Internet, ce qui permet d'aller d'une carte réseau à une autre carte réseau, sur chaque liaison entre deux équipements sur Internet. Les adresses IP ont une autre fonction : elles servent à faire le routage, c'est-à-dire à trouver l'adresse IP du prochain routeur pour atteindre une destination donnée. Une fois l'adresse IP du prochain routeur trouvée, il est nécessaire de trouver l'adresse MAC de la carte réseau du prochain routeur pour lui envoyer le signal.

Ainsi, quand on veut aller d'un client à un serveur sur Internet, on connaît les adresses IP du client et du serveur. Le protocole IP va s'assurer de trouver le chemin grâce aux adresses IP, c'est-à-dire de traverser tous les réseaux intermédiaires qu'il faut traverser. Mais pour traverser ces réseaux, le signal doit circuler sur une ou plusieurs liaisons entre les équipements du réseau. Les adresses MAC sont utilisées sur chaque liaison pour faire que le signal aille de la carte réseau émettrice à la carte réseau destinataire. A chaque fois qu'on va traverser un routeur pour changer de réseau, le routeur connaît l'adresse IP du prochain routeur (c'est le protocole IP qui va la fournir) et ARP va permettre de trouver l'adresse MAC du prochain routeur, c'est-à-dire l'adresse physique de sa carte réseau qui est nécessaire à l'émetteur de la trame pour envoyer le signal.

Pour trouver l'adresse MAC destination (inconnue) correspondant à une adresse IP destination (connue), une requête ARP est diffusée à tous les membres du réseau local pour demander "Qui a cette adresse IP ?". Cette diffusion est cantonnée au réseau local et ne traverse pas les routeurs. L'équipement qui a cette adresse IP renvoie à l'émetteur de la requête ARP une réponse ARP qui contient son adresse MAC.

Le protocole **RARP** (***Reverse Address Resolution Protocol***) permet d'affecter des adresses IP à des cartes réseau lorsqu'elles n'en ont pas. Il est maintenant très peu utilisé et est essentiellement remplacé par le protocole DHCP présenté ci-après.

1.7 Protocole DHCP

DHCP (***Dynamic Host Configuration Protocol***). Ce protocole permet à des machines connectées à un réseau de se voir attribuer une configuration réseau, en particulier une adresse IP, par un serveur DHCP. Ce serveur dispose d'un pool d'adresses qu'il peut mettre à disposition des machines du réseau qui en font la demande. Si vous êtes relié à Internet grâce à une box, elle fait office de serveur DHCP. Quand vous vous connectez au réseau chez vous, vous n'avez rien à configurer, le serveur DHCP de la box s'occupe de vous transmettre la configuration réseau de votre machine ou de votre téléphone pour l'accès à l'Internet.

1.8 Protocoles des cartes réseaux

Les protocoles des cartes réseaux permettent la transmission d'une trame correspondant à une séquence binaire sous la forme d'un signal adapté à une liaison donnée. Les deux protocoles les plus courants sont **Ethernet** et **WiFi**. Les autres protocoles qui sont cités dans l'architecture sont d'autres protocoles de liaison qui ne sont plus tellement utilisés mais qui font partie de l'historique d'Internet. Nous pouvons en revanche ajouter les protocoles de liaisons utilisés dans la téléphonie mobile qui sont les protocoles **3G**, **4G**, **5G**, etc.

Examinons un peu plus en détails le protocole Ethernet. Il permet d'envoyer des signaux électriques à partir d'une carte réseau sur un câble en cuivre.

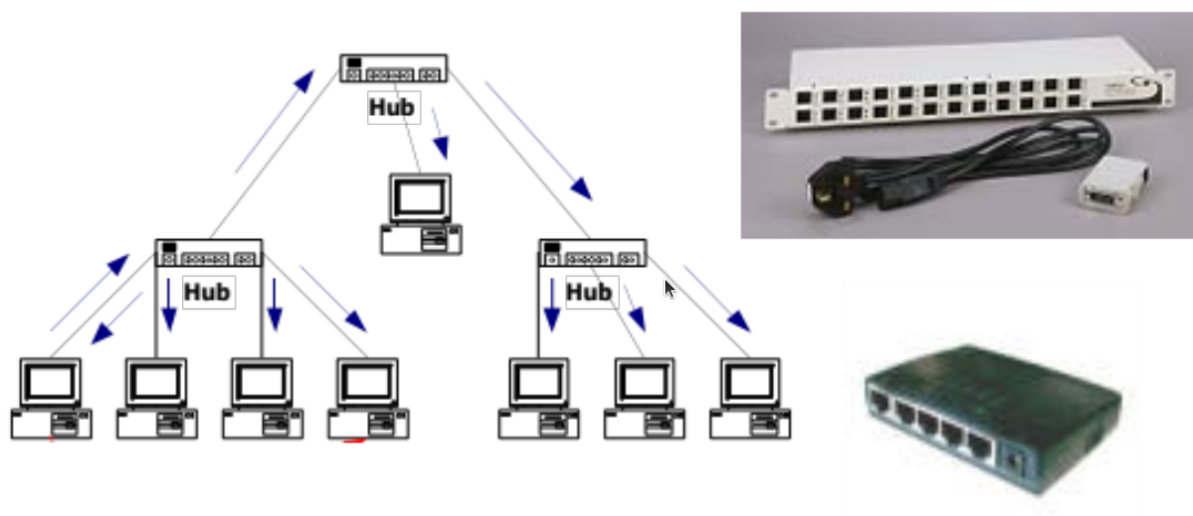


Figure 2: Ethernet

Il y a des équipements d'interconnexion au niveau d'Ethernet qui sont soit des répéteurs, soit des commutateurs. En anglais, c'est le **hub** ou le **switch**.

Ces équipements sont illustrés dans l'image ci-dessus. C'est un ensemble de ports sur lequel des cartes réseau peuvent être branchées. Sur cet exemple, plusieurs ordinateurs en bas sont branchés à travers leur carte réseau et un câble Ethernet sur le hub. Cela permet de former un **réseau local**, c'est-à-dire un ensemble de machines qui sont reliées entre elles et qui peuvent communiquer entre elles. Ici l'équipement intermédiaire est un hub, c'est-à-dire un répéteur. Quand une carte réseau envoie un signal, il arrive au hub qui répète ce signal sur tous ses ports. Le signal est diffusé à tous les ordinateurs du réseau. Le commutateur est un peu plus intelligent que le répéteur : il ne diffuse pas le signal sur tous les ports mais le renvoie uniquement vers la carte réseau destinataire en utilisant son adresse MAC. Pour cela, le commutateur maintient une table d'acheminement qui indique pour chaque port la ou les adresses MAC des cartes réseau qui ont déjà envoyées une trame via ce port.

La trame Ethernet contient dans son entête les **adresses MAC source et destination** des deux cartes réseau émettrice ou destinataire du signal et le champ **type** qui indique ce que contient la trame Ethernet après l'entête. Ce champ type désigne soit IP, soit ARP ; ce sont les deux principaux protocoles qui sont encapsulés dans une trame Ethernet. A la fin de la trame se trouve l'enqueue qui stocke le champ **FCS** (*Frame Control Sequence*) qui est un *checksum* pour faire la détection d'erreur. Il permet d'éliminer toutes les trames qui seraient erronées après avoir traversé une liaison. Le **SFD** *Start Frame Delimitator* (10101011) et le **préambule** (7 fois 10101010) permettent de repérer le début de la trame qui commence à l'adresse MAC destination.

La trame Ethernet a une taille maximale (1518 octets en comptant les 14 octets d'entête et les 4 octets

Trame Ethernet

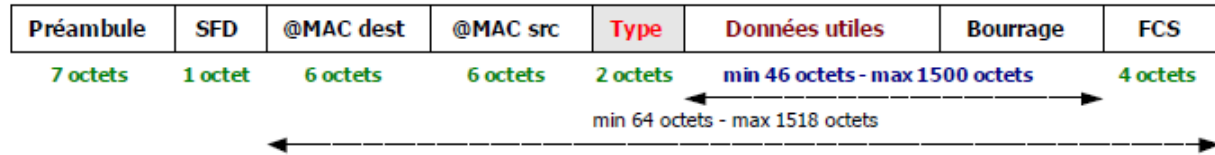


Figure 3: Format Trame Ethernet

d'enqueue) et une taille minimale de 64 octets. Cette taille minimale est nécessaire pour garantir la **détection des collisions**. Ainsi, le contenu de la trame Ethernet (généralement un paquet IP ou ARP) doit faire au minimum 46 octets et au maximum 1500 octets. Si le paquet est plus grand que 1500 octets, IP se charge de le découper en plusieurs paquets de 1500 octets. S'il est plus petit que 46 octets, Ethernet ajoute du **bourrage**, c'est-à-dire des octets artificiels pour atteindre la taille minimale.

1.9 Exemple d'un transfert de fichier

Détaillons un peu plus le fonctionnement de ces protocoles entre eux grâce à un exemple.

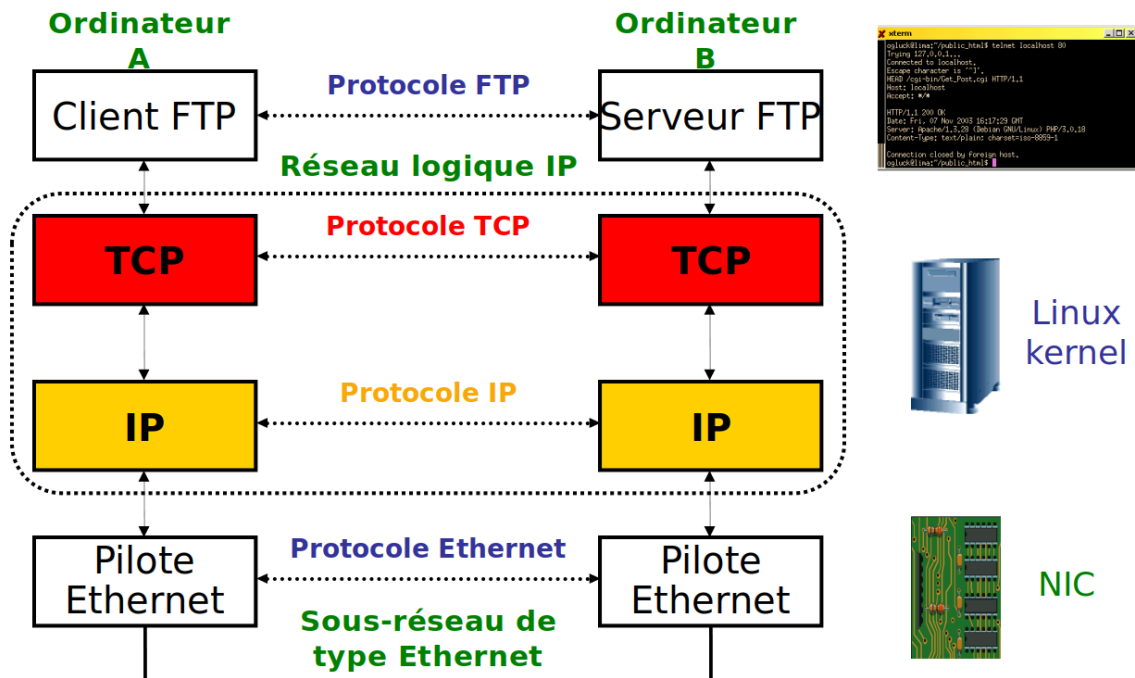


Figure 4: Dialogue FTP sur un réseau local

Sur cette figure, un client FTP dialogue avec un serveur FTP. Les flèches horizontales en pointillés représentent le dialogue entre ces deux entités. C'est à ce niveau que le protocole FTP intervient, dans les échanges entre le client FTP d'un côté et le serveur FTP de l'autre. Le client et le serveur sont chacun un programme qui a été lancé sur un ordinateur. Sur le client, cela peut être un terminal par exemple ou alors un outil spécifique. Le serveur FTP est un programme qui héberge des fichiers et attend des requêtes provenant des clients FTP pour transférer des fichiers depuis le serveur vers le client (**download**) ou inversement (**upload**).

En haut à droite de la figure se trouve le terminal qui correspond au programme Client FTP permettant de faire des requêtes au serveur FTP. Ce dialogue *horizontal* est virtuel. En réalité, la requête FTP qui va permettre de demander un fichier va traverser le réseau en passant par le dialogue *vertical*, c'est à dire en faisant appel aux différents protocoles qui sont nécessaires pour transmettre la requête puis le fichier.

Les protocoles nécessaires ici sont TCP et IP qui se trouvent dans le système d'exploitation de la machine, puis Ethernet dans la carte réseau (NIC pour *Network Interface Card*) de l'ordinateur Client ou Serveur. La requête FTP est donc fabriquée au niveau du Client FTP, puis elle est transmise à TCP sur ce même ordinateur qui va la transmettre à IP qui va la transmettre à la carte réseau. Le signal correspondant à la séquence binaire contenant la requête FTP va partir sur la liaison, c'est-à-dire sur le câble Ethernet qui ici est directement relié à l'ordinateur B. Le signal va être ensuite décodé par la carte réseau de l'ordinateur B, puis transféré (en mémoire) à IP puis à TCP jusqu'à remonter jusqu'au processus du serveur FTP qui va interpréter et traiter la requête.

Donc le dialogue horizontal de chaque protocole se traduit en pratique par un *transfert vertical* à travers les différentes couches de l'architecture TCP/IP jusqu'à la transmission d'un signal sur le câble réseau.

Dans l'exemple ci-dessus, les ordinateurs A et B sont directement reliés entre eux par un câble Ethernet. Ils ne sont pas sur Internet. Pourtant, ils utilisent bien les protocoles de l'architecture TCP/IP qui sont nécessaires pour utiliser l'application de transfert de fichiers FTP. Nous allons maintenant ajouter un équipement intermédiaire, ici un routeur, qui devient nécessaire si le client FTP et le serveur FTP ne sont pas dans le même réseau. C'est bien le cas quand on va sur Internet, on traverse généralement un ou plusieurs réseaux pour atteindre le serveur que l'on souhaite.

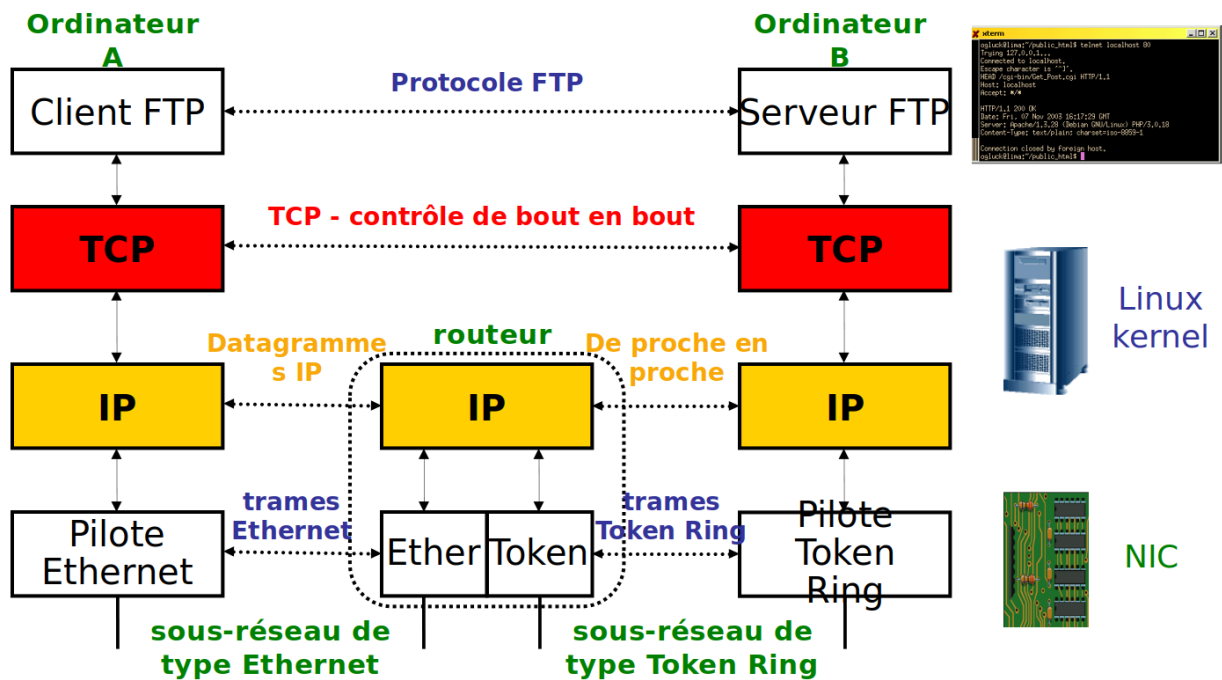


Figure 5: Dialogue FTP à travers différents réseaux

Au milieu de la figure se trouve donc un **routeur**. Un routeur est un équipement intermédiaire qui permet de faire le lien entre deux réseaux sur Internet. Dans l'exemple ci-dessus, le client FTP sur l'ordinateur A se trouve dans un réseau local de type *Ethernet*, typiquement chez vous derrière une box qui joue le rôle de routeur pour aller sur Internet. Elle fait le lien entre le réseau chez vous et le réseau de votre fournisseur d'accès à Internet qui lui-même est relié à d'autres réseaux. Le serveur FTP se trouve sur l'ordinateur B

dans un autre réseau local, ici de type *Token Ring*. Le routeur au milieu fait le lien entre les deux réseaux grâce à deux cartes réseau de type différent : une carte réseau Ethernet qui permet de communiquer avec le réseau du client FTP et une carte réseau Token Ring dans le réseau du serveur. Le routeur est donc nécessaire pour interconnecter le réseau Ethernet du client avec le réseau Token Ring du serveur et assurer la transmission des requêtes et des réponses d'un réseau à l'autre.

Sur la figure, vous pouvez constater que les protocoles FTP et TCP ne sont présents qu'aux extrémités, c'est-à-dire là où il y a le client et le serveur. On dit que ces protocoles sont de **bout en bout**. Ils ne s'exécutent pas sur les équipements intermédiaires, comme le routeur. À l'opposé, les autres protocoles comme IP et Ethernet sont **point à point**. Ils s'exécutent sur chaque liaison et donc sur chaque ordinateur mais aussi sur le routeur.

Il y a un vocabulaire spécifique pour désigner les différents messages qui circulent. Au niveau applicatif, ici FTP, on parle de requête ou réponse selon que le message va du Client vers le Serveur ou inversement. Au niveau IP, on parle de **datagramme** ou de **paquet IP**. Au niveau de la liaison, on parle plutôt de **trame**.

1.10 L'encapsulation

Nous devons maintenant décrire comment les messages sont fabriqués au niveau de chaque protocole. C'est ce qu'on appelle l'**encapsulation**.

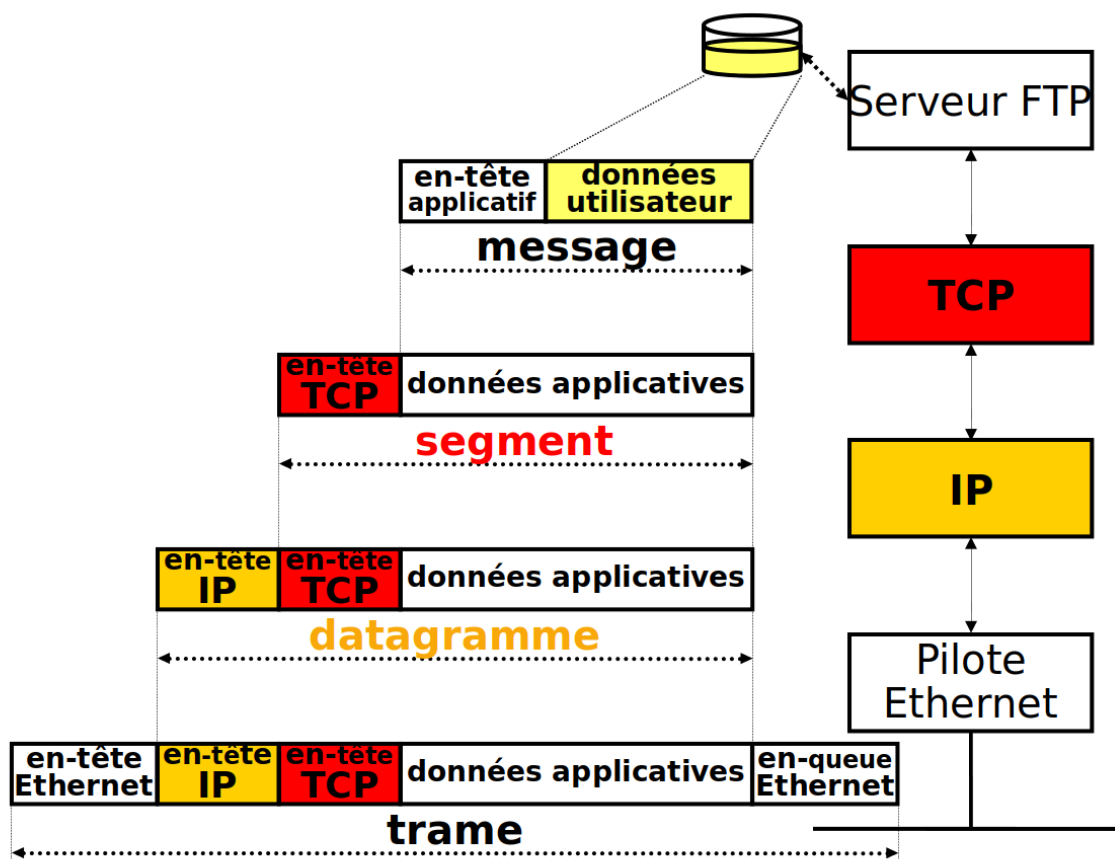


Figure 6: Encapsulation

Considérons la situation suivante : un fichier ou un morceau de fichier est transféré du serveur FTP vers le client FTP. Ce fichier correspond aux *données utilisateur* c'est-à-dire à ce que l'utilisateur de l'application

souhaite transmettre. Il s'agit de la *charge utile*.

On veut donc transmettre un fichier qui se trouve sur le disque dur du serveur FTP. En plus de la transmission du fichier, le serveur FTP va ajouter des informations complémentaires dans un en-tête qu'il va fabriquer. Ces informations complémentaires sont destinées au client FTP pour qu'il puisse interpréter correctement les données reçues. Par exemple, le serveur FTP doit indiquer le mode de transfert du fichier qui peut être de type *binaire* ou *ascii* s'il s'agit d'un fichier texte. Ce message qui contient l'en-tête et le fichier est transmis à TCP, qui lui aussi a besoin d'ajouter un en-tête pour assurer son rôle, typiquement la fiabilité. Par exemple, l'en-tête TCP doit contenir une somme de contrôle pour faire la détection d'erreurs, mais aussi les numéros de port permettant d'identifier le client et le serveur FTP. Ce message augmenté de l'en-tête TCP est ensuite transmis à IP qui ajoute dans son en-tête, en particulier, les adresses IP source et destination permettant au routeur d'acheminer chaque paquet vers la bonne destination. Enfin, la carte réseau construit la trame Ethernet en ajoutant son en-tête qui contient les adresses MAC et son en-tête qui contient une somme de contrôle de la trame. Ainsi, l'encapsulation qui consiste en l'ajout par chaque protocole d'informations qui lui sont propres est terminée. Au niveau du client, après réception de la trame, le processus inverse va s'opérer : chaque couche va "éprouver" le message reçu, extraire et lire l'en-tête qui lui est destiné, puis fournir à la couche supérieure le reste du message, c'est-à-dire les données qui lui sont destinées. Au final, le client FTP ne récupérera que son en-tête et les données utilisateurs, c'est-à-dire le fichier transféré.

Donc, l'**encapsulation** consiste à inclure les données issues d'un protocole de haut niveau dans les données d'un protocole de plus bas niveau. Ainsi, chaque couche ajoute les informations lui permettant de rendre son service.

1.11 Fonctionnement d'Internet

Il nous reste maintenant à voir quelques précisions sur les protocoles qui sont au cœur de l'architecture, à savoir les protocoles IP et TCP.

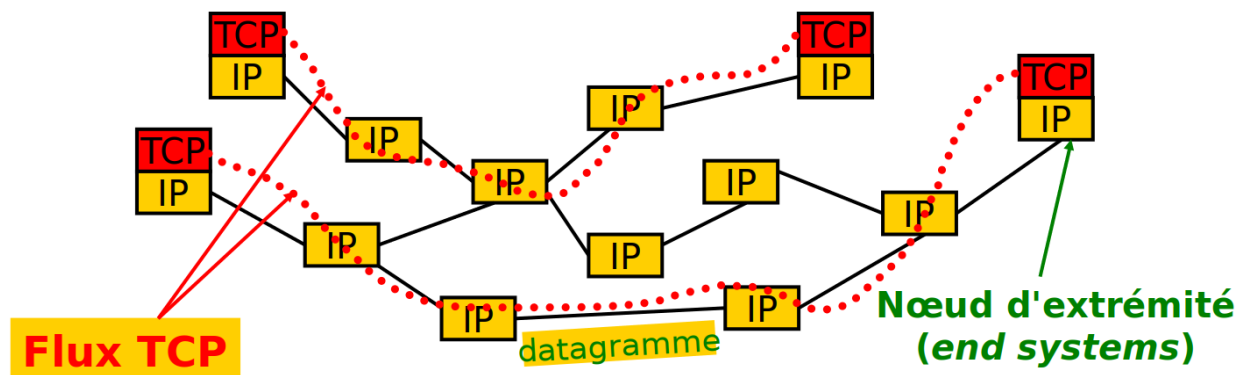


Figure 7: Fonctionnement d'Internet

Dans le schéma ci-dessus, chaque boîte représente un équipement sur Internet qui exécute le protocole IP. Donc, ça peut être soit des clients ou des serveurs, soit des routeurs intermédiaires.

Le rôle du protocole IP est d'acheminer les paquets (ou *datagrammes*) sur Internet. Il doit être rapide et simple car il s'exécute sur tous les équipements intermédiaires sur Internet. En effet, si on ne veut pas créer des embouteillages, il faut que ça aille le plus vite possible. D'une certaine manière, chaque équipement peut être vu comme un carrefour avec des feux tricolores et les paquets doivent rester le moins longtemps possible à chaque carrefour. En particulier, ils doivent très rapidement décider quelle liaison prendre.

Le protocole IP est en mode **non connecté**. Cela signifie que, quand un routeur envoie un paquet IP au routeur suivant, il ne va pas lui demander son accord avant de le faire. Il n'y a donc pas une phase de

connexion entre les deux routeurs avant l'envoi du paquet. Le mode non connecté permet d'aller plus vite.

Le protocole IP s'exécute rapidement aussi parce qu'il est simple. Mais en contrepartie, il n'apporte aucune garantie sur la bonne arrivée des messages. On dit aussi qu'IP fait du **best effort**. Cela signifie qu'il fait au mieux mais sans garantie. Par exemple, IP ne détecte pas les pertes de paquets. Qui va apporter cette garantie si une application en a le besoin ? C'est le protocole TCP qui assure la fiabilité des transferts de bout en bout.

Enfin, le protocole IP est **robuste**. Cela signifie qu'il peut continuer à fonctionner même en cas de panne d'un équipement intermédiaire. Par exemple, si un des routeurs intermédiaires tombe en panne, IP peut, sous certaines conditions, détecter automatiquement la disparition du routeur et modifier les tables de routage pour réorienter le trafic vers d'autres routeurs. Le protocole IP est robuste car il s'adapte aux changements de route, en cas de panne de certaines liaisons ou certains équipements. Nous verrons que cela peut se faire à condition d'avoir du routage dynamique.

Le protocole TCP fait le lien avec les applications et assure la fiabilité, c'est-à-dire qu'il fait en sorte que tout ce qui est envoyé arrive au destinataire final, sans erreur, sans perte, sans double et dans l'ordre.

Dans le schéma ci-dessus, nous avons représenté deux **connexions TCP**, chacune correspondant à un chemin en pointillés. Nous parlons de connexion car le protocole TCP est en mode connecté, contrairement à tous les autres protocoles de l'architecture TCP/IP. Le mode connecté est important pour assurer la fiabilité. Avant de commencer les échanges entre un client et un serveur, le client fait une demande d'ouverture de connexion vers le serveur pour savoir s'il est d'accord et prêt pour échanger. Le serveur accepte ou non la connexion. Une fois que la connexion est acceptée, le client et le serveur peuvent s'échanger des données. Le fonctionnement est le même que pour une communication téléphonique. Quand vous appelez quelqu'un, ça sonne. Il a le choix, soit il répond, soit il ne répond pas. S'il vous répond, vous pouvez dialoguer le temps que vous souhaitez jusqu'à ce que l'une ou l'autre des parties décide de fermer la connexion, c'est-à-dire de raccrocher.

Une fois la connexion établie, le protocole TCP fait transiter un flux d'octets de manière fiable entre le client et le serveur se trouvant aux extrémités de la connexion représentée par les pointillés. TCP est donc bien de **bout en bout**. Sur ce schéma, deux connexions sont ouvertes. Par exemple, une connexion entre un navigateur Web et un serveur Web en bas et une autre en haut entre un client FTP et un serveur FTP. Il faut s'imaginer que ces pointillés peuvent bouger pendant la durée de vie de la connexion, potentiellement en plein milieu du transfert d'un fichier. En effet, il se peut qu'un routeur tombe en panne au milieu d'Internet et que les paquets d'une même connexion empruntent des chemins différents du fait d'une modification des tables de routage pour prendre en compte cette panne.

Le protocole TCP doit assurer la fiabilité de la connexion au-dessus d'un réseau non fiable. C'est un protocole assez complexe, puisqu'il doit détecter les erreurs et les pertes, gérer les retransmissions le cas échéant et remettre les paquets dans l'ordre s'ils arrivent dans le désordre, c'est-à-dire dans un ordre différent de l'envoi.

Regardons maintenant les différents protocoles dans l'architecture TCP/IP et comment ils font appel les uns aux autres à la réception, c'est-à-dire comment chaque protocole va désigner le protocole supérieur à qui il doit transmettre les données.

Comme nous l'avons vu précédemment, l'encapsulation consiste à ajouter un en-tête par chaque protocole côté émetteur. Cet en-tête est transmis dans le réseau pour être lu par le récepteur de même niveau que l'émetteur. Les informations contenues dans l'en-tête permettent au protocole de faire son travail et de désigner le protocole suivant. Ainsi, quand les données arrivent à destination, elles sont reçues par la carte réseau qui doit transmettre le contenu de la trame soit à IP, soit à ARP, soit à RARP. Il faut ensuite remonter dans l'architecture jusqu'à délivrer le message à la bonne application. Il s'agit de la **désencapsulation** : chaque protocole, en partant du bas, identifie chaque protocole qui se trouve au-dessus de lui. Par exemple, dans le cas d'une requête FTP encapsulée dans TCP lui-même encapsulé dans IP puis dans Ethernet, il faut dans l'en-tête de chaque protocole désigner le protocole encapsulé, c'est-à-dire celui qui se trouve au-dessus. Ainsi, dans l'entête Ethernet, il y a un champ qui s'appelle **EtherType** qui permet d'identifier le protocole qui peut se trouver au-dessus. Que peut contenir une trame dans une carte réseau ? Et bien c'est

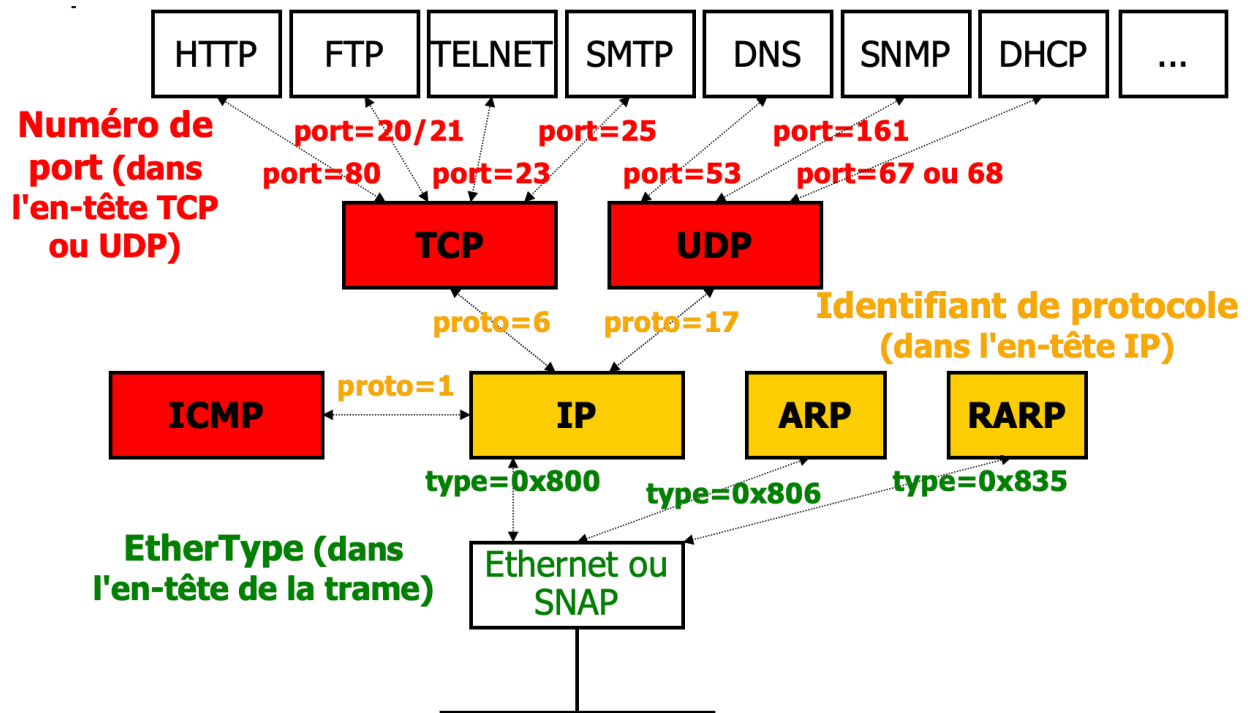


Figure 8: Identification des protocoles

soit un paquet IP (EtherType=0x0800), soit un paquet ARP (EtherType=0x0806), soit un paquet RARP (EtherType=0x0835). Ce sont des valeurs en hexadécimales sur deux octets.

De la même manière, dans l'entête IP, il y a un champ qui s'appelle **proto** qui permet d'identifier le contenu du paquet IP, c'est-à-dire soit ICMP (proto=1), soit TCP (proto=6), soit UDP (proto=17). Enfin, au niveau de TCP ou UDP, c'est le **numéro de port** qui permet d'identifier quel est le programme client ou serveur et donc l'application qui est à l'origine du message échangé. Les numéros indiqués sur la figure correspondent aux numéros de ports des serveurs qui sont fixés à l'avance par la norme : 80 pour le serveur HTTP, 25 pour SMTP, etc.

Voilà donc comment les messages sont acheminés depuis l'arrivée dans la carte réseau vers le bon endroit, c'est-à-dire la bonne application qui s'exécute sur la machine.

Le protocole ICMP a une particularité importante à noter. Il est encapsulé dans IP mais si on revient sur l'architecture TCP/IP, il se situe au même niveau qu'IP dans le modèle OSI (couche 3). Du point de vue de l'encapsulation, il est au niveau 4 comme TCP et UDP. Mais puisqu'il permet de faire des tests en différents points du réseau, il doit s'exécuter sur les équipements intermédiaires que sont les routeurs et il est point-à-point comme IP. De ce point de vue, il fait partie de la couche réseau du modèle OSI.

1.12 Numéros de port

Revenons un peu sur ces numéros de port. Nous avons vu un peu plus haut qu'il y a des connexions TCP entre un client et un serveur (représentées par les pointillés rouges). Ce qui permet d'identifier une connexion, c'est évidemment l'adresse IP du client et du serveur, mais aussi le numéro de port du client et celui du serveur.

Ce qu'on appelle une **adresse de transport** (ou **adresse de socket**), c'est une adresse IP plus un numéro de port (+ un bit pour savoir si on utilise TCP ou UDP) qui permet d'identifier de manière unique le point de communication local (c'est-à-dire la socket) utilisé par le client ou le serveur. Une **connexion TCP** identifie de manière unique une communication entre un client et un serveur qui utilisent TCP. Elle est donc

caractérisée par un **couple d'adresse de transport**.

Une connexion TCP doit être identifiée de manière unique. Sur Internet, comment garantir cela ? Par la manière dont on attribue le numéro de port d'un client ou d'un serveur permettant de garantir l'unicité de la connexion. Les ports des serveurs doivent être fixés pour que les clients puissent se connecter à eux. Comme c'est le client qui initie la communication, il doit connaître le numéro de port du serveur pour pouvoir lui envoyer le premier message. En revanche, le port d'un client n'a pas besoin d'être fixé à l'avance.

Pour les serveurs, leur numéro de port est défini par la norme et connu de tous. Les numéros de port des serveurs sont ceux qui sont **inférieurs à 1024**. On parle aussi de **ports réservés**. Cela correspond aux ports que nous avons déjà listés et qui apparaissent sur l'architecture TCP/IP : 80 pour le web, 25 pour le mail, etc.

Pour les clients, leur numéro n'est pas fixé à l'avance. Il est attribué au moment où la communication est initiée par lui. Chaque client utilise un **port dynamique** c'est-à-dire un port supérieur à 1024 et inférieur à 65536 parmi ceux qui sont disponibles. Un numéro de port étant codé sur deux octets, sa valeur maximale est 65535. Par contre, il faut garantir, que sur une même machine, il n'y a pas deux clients qui utilisent le même numéro de port. C'est le système d'exploitation qui garantit cette unicité.

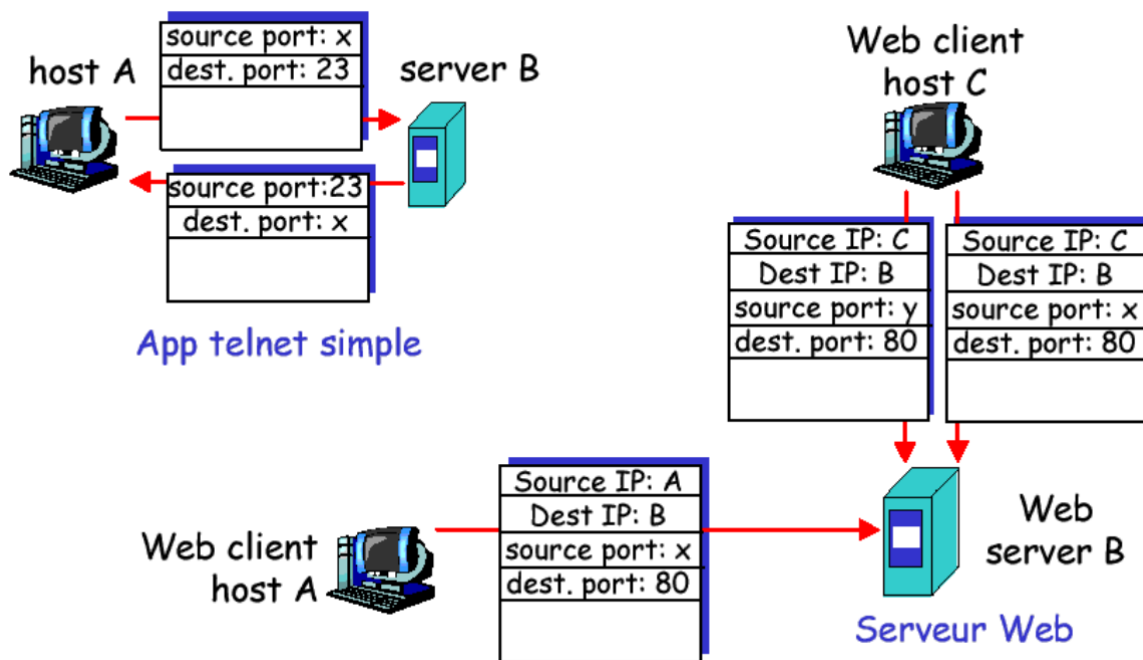


Figure 9: Exemple de mutliplexage à l'aide des numéros de port

Considérons l'exemple ci-dessus. Vous avez en haut à gauche une connexion entre une machine A et une machine B. Le client sur la machine A et le serveur sur la machine B. Ici, il s'agit d'une connexion entre un client et un serveur **Telnet**. Pourquoi ? Parce que le port destination est 23 (port réservé au serveur Telnet). Le port source utilisé par le client Telnet est le port x, x signifiant un port dynamique, c'est-à-dire un port alloué par le système d'exploitation de la machine cliente (machine A). x est donc un nombre supérieur à 1024 et inférieur à 65536.

Quand le client fait sa demande de connexion TCP, le port destination est 23 et le port source est x. Quand le serveur répond, on inverse la source et la destination. Le port source devient 23 et le port destination est x. Ces deux informations vont circuler dans l'entête TCP (ou UDP selon que l'application utilise TCP ou UDP).

Sur l'autre exemple, un serveur web se trouve hébergé sur la machine B située en bas à droite. En effet,

l'application sur la machine B utilise le port 80 qui est réservé au serveur HTTP. Deux machines clientes différentes, c'est-à-dire deux machines physiques différentes avec des adresses IP différentes A et C exécutent un ou plusieurs navigateurs web pour faire des demandes à ce serveur web.

Chaque client HTTP utilise un port source x ou y , selon le client. Les adresses IP sources distinguent les trois machines A, B et C. Les connexions sont bien toutes distinguables soit par un numéro de port source ou destination différent soit par une adresse IP source ou destination différente. Ici, ce qui distingue ces 3 connexions, c'est l'adresse IP source combinée au port source. Sur la machine C, il y a deux connexions distinctes qui correspondent à deux navigateurs web différents faisant une demande au même serveur web, celui de la machine B. Les réponses du serveur vont bien être acheminées vers le bon navigateur du fait que chaque navigateur utilise un port différent, d'où la nécessité de garantir x différent de y .